

PublicPolls

Sistema de Questionários Online

Instituição: INFNET

Disciplina: Arquitetura .NET

Professor: Ubirajara Mendes Nunes Junior

Aluno: Gabriel Bezerra Rodrigues

Email: gabriel.brodrigues@al.infnet.edu.br

Repositório GitHub:

<https://github.com/gabrigabs/public-pools-app>

Dezembro de 2024

Sumário

1. Visão Geral

2. Estrutura do Projeto

3. Diagrama de Contexto (C4)

4. Diagrama de Containers (C4)

5. Modelo de Dados

6. Escalabilidade

7. Como Executar (Docker e Local)

8. Conformidade Acadêmica

1. Visão Geral

O **PublicPolls** é uma solução de arquitetura moderna baseada em .NET 8 para gestão de pesquisas eleitorais de alta escala. O projeto utiliza Blazor WebAssembly no frontend e uma API REST ASP.NET Core no backend, garantindo performance e experiência de usuário fluida.

Documentação Completa: Para detalhes aprofundados sobre cada decisão arquitetural, consulte a pasta `docs/` na raiz do repositório.

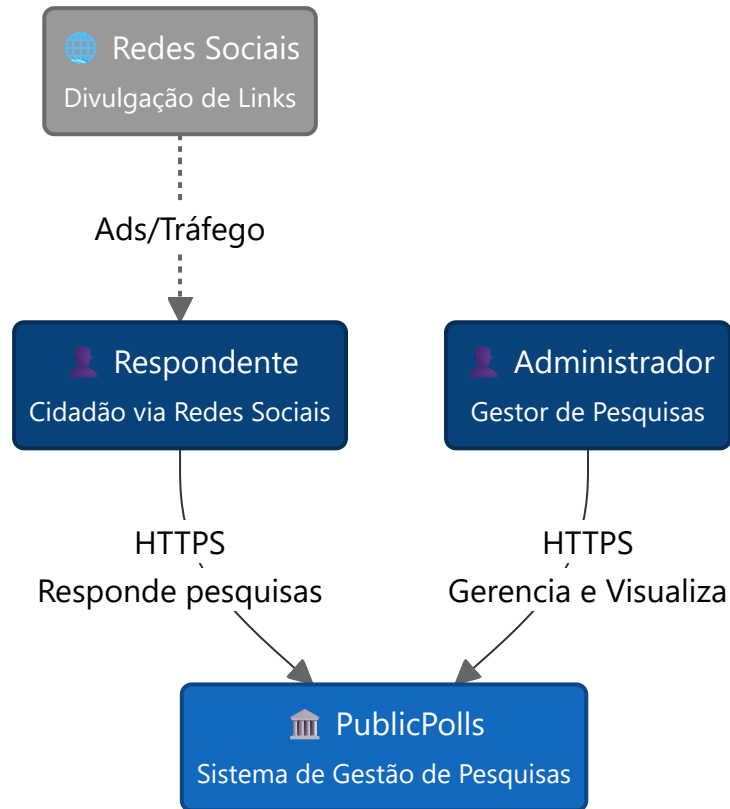
2. Estrutura do Repositório

O projeto segue uma estrutura organizada separando documentação, código fonte e testes:

```
public-pools-app/
├── docs/                    # Documentação detalhada do projeto
│   ├── 01-visao-geral.md   # Contexto e Requisitos
│   ├── 02-arquitetura-c4.md # Diagramas Arquiteturais
│   ├── 03-modelo-dados.md  # Modelagem do Banco
│   ├── 04-justificativas.md # Defesa Técnica das Escolhas
│   ├── 05-escalabilidade.md # Plano de Escala
│   └── 06-api-reference.md  # Docs da API
├── src/                    # Código Fonte
│   ├── PublicPolls.API/    # Backend (ASP.NET Core)
│   ├── PublicPolls.Web/    # Frontend (Blazor WASM)
│   ├── PublicPolls.Domain/ # Regras de Negócio e Entidades
│   ├── PublicPolls.App/    # Camada de Aplicação (Services)
│   └── PublicPolls.Infra/   # Acesso a Dados (EF Core)
├── tests/                  # Testes Automatizados
└── docker-compose.yml      # Orquestração de Containers
```

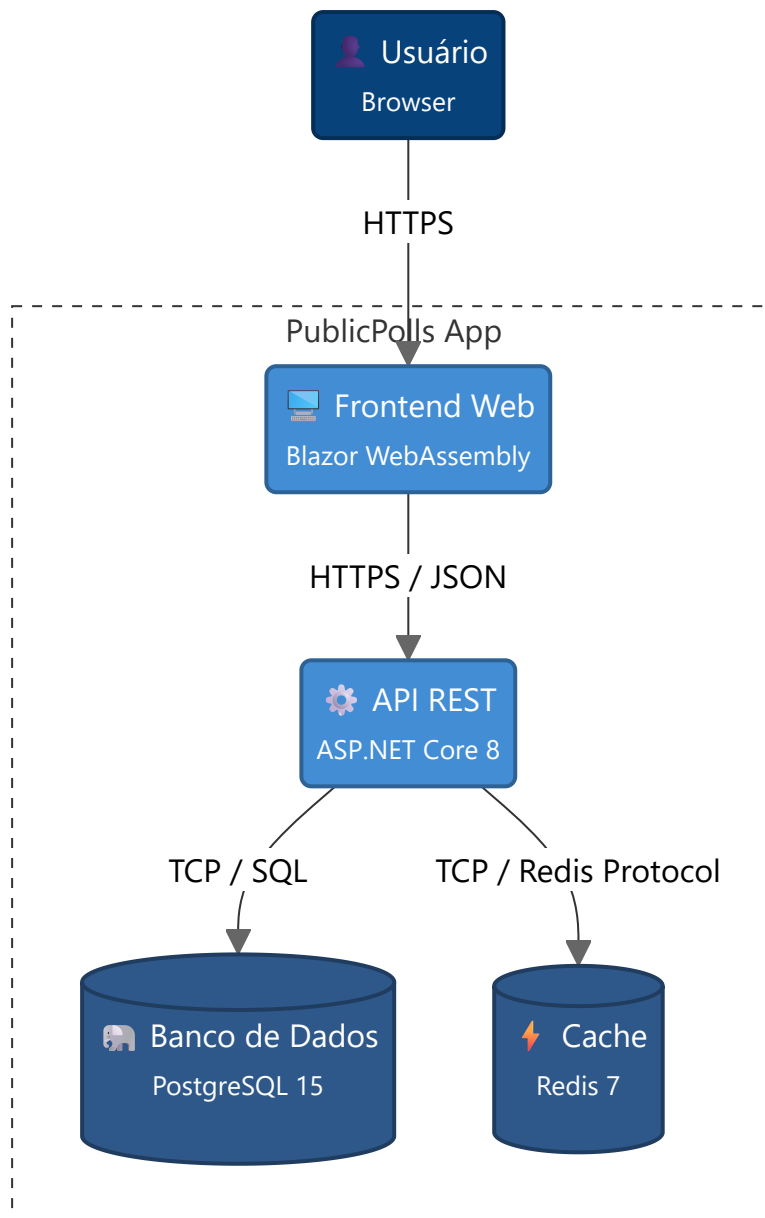
3. Diagrama de Contexto (C4 Nível 1)

Visão macro do sistema e suas interações externas.



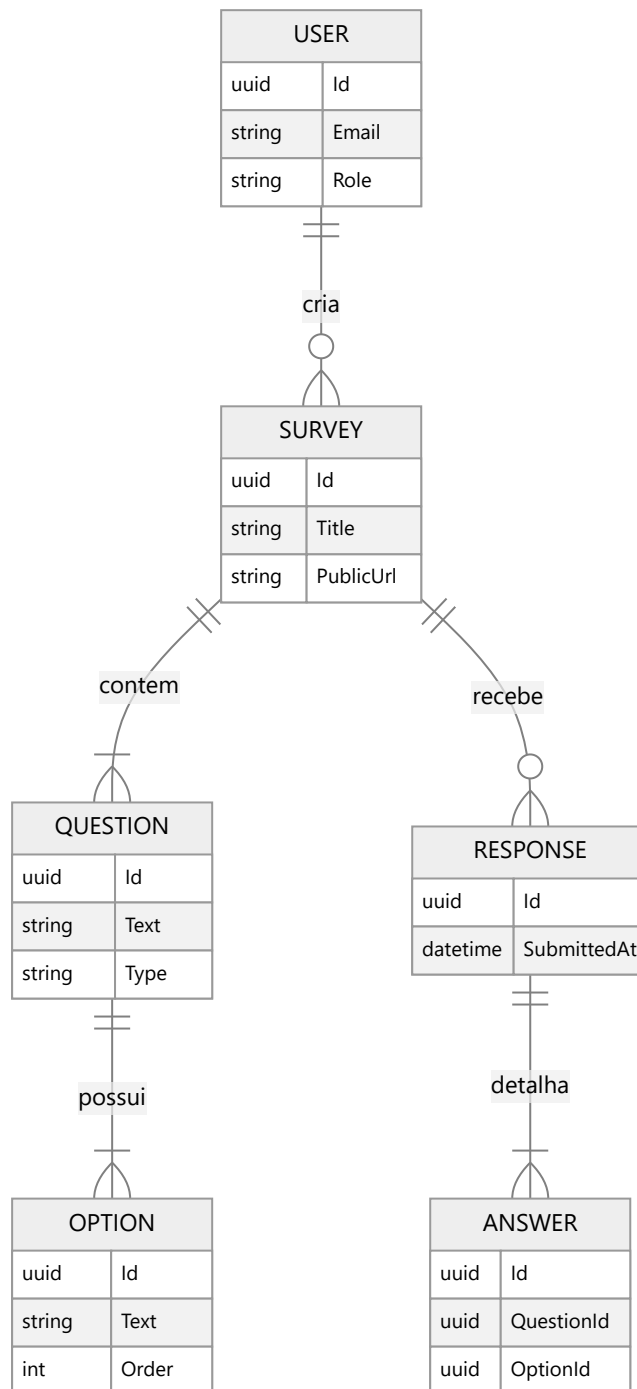
4. Diagrama de Containers (C4 Nível 2)

Detalhamento dos componentes implantáveis do sistema.



5. Modelo de Dados

O diagrama abaixo ilustra as principais entidades e relacionamentos do sistema.



6. Escalabilidade

A arquitetura foi desenhada para alta performance:

- **Cache-First:** O Redis intercepta leituras de pesquisas públicas, reduzindo a carga no banco em 90%.
- **Stateless API:** Autenticação JWT permite escalar instâncias da API horizontalmente sem problemas de sessão.
- **Otimização de Banco:** Índices em colunas críticas (`PublicUrl`, `SubmittedAt`) garantem consultas rápidas.

7. Como Executar o Projeto

Opção A: Com Docker (Recomendado)

A maneira mais simples de rodar todo o ambiente (Banco, Cache, API e Frontend).

```
# Na raiz do projeto
docker-compose up -d --build
```

Após iniciar:

- **Frontend:** <http://localhost:5002>
- **API Swagger:** <http://localhost:5001/swagger>

Opção B: Sem Docker (Manual)

Caso prefira rodar as aplicações diretamente na máquina host. Você ainda precisará de instâncias de PostgreSQL e Redis rodando.

1. Pré-requisitos

- .NET 8 SDK instalado
- PostgreSQL rodando (porta 5432)
- Redis rodando (porta 6379)

2. Executar a API

```
cd src/PublicPolls.API
# Ajuste o appsettings.json se seu banco tiver senha diferente
dotnet run
# A API iniciará em https://localhost:5001
```

3. Executar o Frontend

```
cd src/PublicPolls.Web
dotnet run
# O Frontend iniciará em https://localhost:5002
```